

VoiceChess

Project Report

A voice controlled Cartesian robot for physical chess

Technical University of Munich

TUM Campus Heilbronn

Embedded Systems, Cyber-Physical Systems and Robotics

INHN0018 · Summer semester 2026

September 2026



VoiceChess prototype with Cartesian gantry, vertical gripper and physical chessboard.

Abstract

VoiceChess combines an offline speech interface, a chess engine and a Cartesian gantry to move physical chess pieces in response to spoken commands. A Raspberry Pi 5 interprets and validates moves, while an ESP32 running FluidNC executes the generated motion commands. We built the mechanism from aluminium profiles, belt drives and custom PETG parts, using a parallel gripper to handle the pieces. The integrated prototype supports voice-controlled play, captures and castling. En passant and promotion are implemented in software but have not been tested on the robot. This report explains the design decisions, hardware and software interfaces, calibration, demonstrated results and remaining validation work.

Team and contributions

Recorded contributions

The following contributions are recorded in the project presentation. This table identifies the documented technical work and does not determine course membership.

Contributor	Recorded responsibility
Matei-Florian Tudose	Team co-lead, electronics
David-Luca Mosu	Team co-lead, mechanics and design
Anton Stoyanov	Software
Eni Ferizi	Software
David Enachescu-Goldenberg	Software
David Gabriel-Marin	Software
Dorde Vidakovic	Software

1 Project objectives and requirements

Moving pieces by hand is a barrier to playing conventional physical chess for people with limited hand mobility. Our objective was to let a player specify moves by voice while retaining a real board and pieces. The robot executes both the human move and the computer reply, so routine play does not require the player to reach across the board. Accessibility is the motivation for the project; no formal usability study with the intended user group has been conducted.

The semester project had a target purchasing budget of €700 and access to shared 3D printers and basic workshop tools. The mechanical design therefore relies on purchased rails, wheels, motors and fasteners for standard functions, with printed parts for geometry specific to the robot.

Requirement	Design response
Spoken move entry	Offline speech recognition, legal-move matching and spoken confirmation
Physical piece handling	Three translation axes and a servo-driven parallel gripper
Correct chess behaviour	A single rules engine supplies legal moves and maintains the position
Computer opponent	Stockfish selects the reply at configurable difficulty
Affordable construction	Standard components and PETG parts printed using shared facilities
Develop without the robot	Text input, scripted moves, serial dry run and automated tests

Scope of the prototype

The system assumes that the physical position matches the software board, that the board is aligned with the machine coordinates, and that pieces have the dimensions used during calibration. It does not use a camera or encoders to verify a successful grasp or detect a displaced piece. Initial setup and recovery from a physical fault therefore require an operator. Hands-free move entry should not be confused with unattended operation.

Report basis

The report combines the project presentation, the mechanical design report and the VoiceChess repository at commit c4a1d12, with final hardware and demonstration details confirmed by the team. Software configuration values are identified as such and are not treated as independent physical measurements.

2 System architecture

The architecture separates game-level decisions from real-time motor control. A single Python process on the Raspberry Pi 5 coordinates speech recognition, chess rules, move planning and serial communication. The ESP32 receives G-code over USB and produces step, direction and servo signals through FluidNC. This keeps speech and chess processing out of the timing-sensitive pulse-generation layer.

Layer	Responsibility	Interface
Speech interface	Recognize and confirm the intended move	Microphone input and spoken feedback
Chess engine	Validate moves and obtain the computer reply	UCI move strings and board state
Motion planner	Translate each chess move into physical transfers	G-code lines
Serial link	Send commands and wait for completion	USB serial at 115200 baud
Motion controller	Generate step pulses and servo PWM	ESP32 GPIO to drivers and gripper

Sequence of one turn

The orchestrator obtains the legal moves for the current position and supplies them to the voice module. The player speaks a move; ambiguous or unrecognized input is requested again. After spoken confirmation, the chess engine validates the move, produces a readback and updates its internal board. The planner uses the pre-move piece information to generate the corresponding transfers. The serial link sends the commands and waits for the controller to become idle before the computer reply is planned and executed.

The program returns to listening only after the robot has finished the current command sequence. This distinction matters because FluidNC's "ok" response acknowledges acceptance into its command pipeline, not completion of the physical motion. The serial layer polls the controller state until it reports "Idle".

Module boundaries

The repository divides the Python application into `voice_matching`, `chess_ai`, `motion` and `orchestrator` packages, with `main.py` providing configuration and startup. Each role exchanges small, explicit values rather than directly controlling the other modules. For example, the chess engine produces UCI strings such as `e2e4`, while the motion planner needs the moving piece type, any captured piece and the positions of tall obstacles. This arrangement supports unit tests without a microphone or powered gantry.

The physical and digital state are coupled by successful execution, rather than by sensor feedback. A serial failure or dropped piece can break that relationship. The current implementation updates the digital board before the physical transfer and does not automatically roll it back after a motion fault; recovery remains an explicit limitation.

3 Mechanical structure and actuation

We selected a Cartesian gantry because a chessboard maps naturally to two horizontal axes, while an independent vertical axis can lift pieces clear of their neighbours. An under-board magnetic dragging concept was considered, but it would have required a different strategy for obstacle clearance, captured pieces and replacement pieces. The overhead gripper provides a direct way to remove and place pieces.

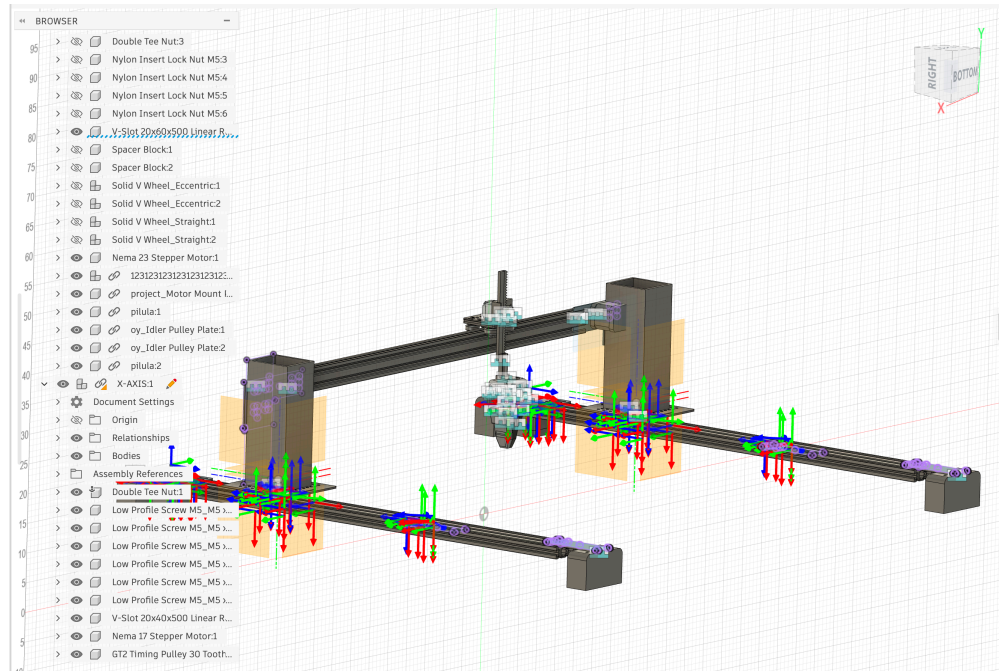


Figure 1 CAD assembly of the gantry. The two parallel rails support the moving bridge.

Frame and guidance

Two 800 mm aluminium profiles form the parallel rails, with an 800 mm bridge spanning the board. The mechanical report specifies 20×60 mm rail profiles and a 20×40 mm bridge profile. V-wheels run in the profile grooves, and eccentric spacers allow the wheel preload to be adjusted. Printed mounts connect the profiles to metal towers, with hole patterns providing adjustment during alignment. The table serves as the supporting base.

Horizontal drives

Three NEMA 23 stepper motors drive the horizontal motion through 6 mm GT2 belts. Two motors move the parallel sides of the gantry and are commanded together; the third drives the carriage along the bridge. In the current software convention, X follows the parallel rails from a1 towards a8, and Y follows the bridge from a1 towards h1. Earlier design material uses different axis labels, so this report uses the repository convention throughout.

Ganged step commands keep the two rail drives synchronized electrically, but they do not measure whether either motor has lost steps. Rail alignment, equal belt tension and adequate torque remain important to prevent skew. The printed belt fixtures and adjustable wheel mounts make these mechanical corrections possible without remanufacturing the frame.

4 Vertical axis and gripper design

A geared NEMA 17 stepper drives the printed rack-and-pinion vertical axis. The approximately 5:1 reduction increases available lifting torque. The final build uses a 76 mm king and a confirmed 115 mm working Z travel. In the software frame, Z0 is the upper position and increasing Z moves the gripper down; the board plane is Z115. The repository also contains a 170 mm full-axis envelope, which is a configuration value rather than a replacement for the confirmed working travel.

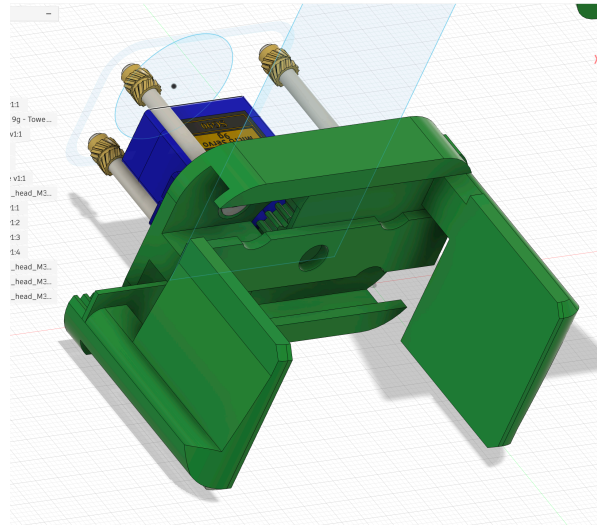


Figure 2 Parallel gripper CAD model with opposed sliding jaws.

Gripper iteration

The first claw concept used pivoting fingers. Its swept volume risked contact with adjacent pieces, which made it unsuitable for a crowded board. The revised mechanism uses an SG90 servo to drive two opposed racks, keeping the jaws parallel as they close. The mechanical design describes an opening of approximately 70 mm. The planner uses separately calibrated dimensions and closure angles, rather than assuming the full mechanical opening is required for each move.

Piece-specific pickup depths and servo angles allow the same gripper to grasp pieces with different heights and profiles. A fixed dwell after each opening or closing command gives the servo time to move before the next transfer. Grip success is not sensed, so reliable operation depends on consistent piece geometry and repeatable placement.

Materials and manufacture

Custom mounts, carriage components, racks and gripper parts were printed in PETG. The mechanical report specifies 60% triangular infill for structural parts and 20% for less loaded parts, with brass heat-set inserts where repeated assembly would otherwise damage the plastic threads. Its estimate that roughly 65% of the design is printed describes the construction approach; it is not a measured mass or cost fraction.

5 Electronics and power distribution

The electronics connect the Raspberry Pi's game-level control to four stepper drivers and the gripper servo. A 24 V, 10 A supply feeds the motor-power distribution through a fuse, while a buck converter supplies the servo at approximately 4.8 V. The Raspberry Pi has its own power supply and powers the ESP32 through the USB connection. The grounds are connected at the distribution and controller wiring, providing a shared reference for control signals.

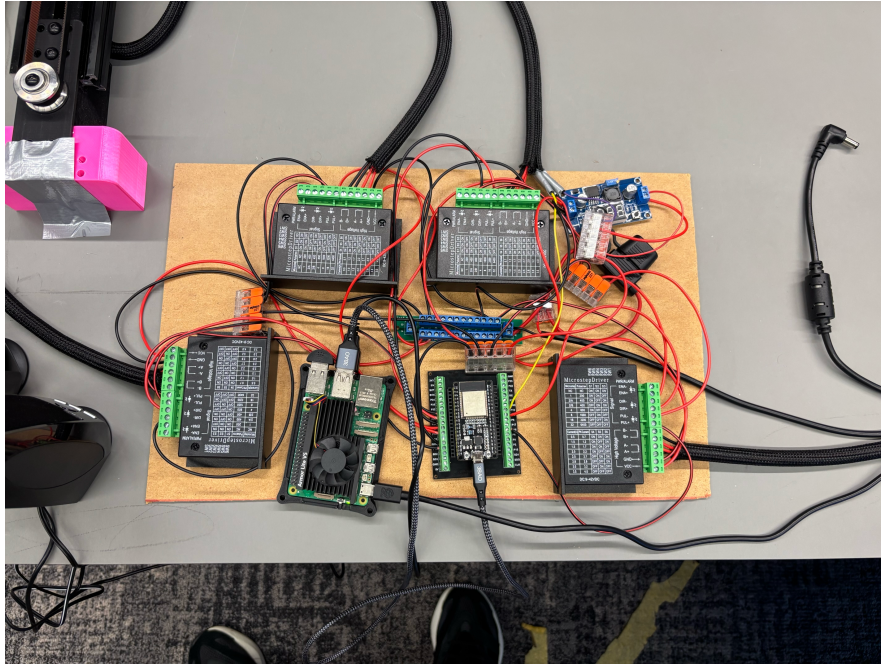


Figure 3 Prototype electronics with four TB6600 drivers and power distribution.

Component	Role
Raspberry Pi 5	Speech processing, chess engine, planning and orchestration
ESP32 with FluidNC	Real-time motor and servo command execution
Four TB6600 drivers	Two X motors, one Y motor and one Z motor
SG90 servo and buck converter	Parallel gripper actuation on a low-voltage rail
USB microphone and speakers	Voice input, confirmation and move readback
24 V 10 A supply	Motor supply with a nominal output rating of 240 W

The installed fuse is rated at 10 A. The team confirms driver current settings of 2.5 A for each NEMA 23 and 0.5 A for the geared NEMA 17. These phase-current settings cannot be added directly to the servo current to establish 24 V input demand, because the drivers regulate winding current and the servo uses a converter.

6 Embedded control and calibration

The committed FluidNC configuration uses the ESP32 RMT stepping engine, a 6 μ s step pulse and a 5 μ s direction delay. All four drivers are configured on the software side for 1/16 microstepping, equivalent to 3200 pulses per motor revolution for a 200-step motor. The physical driver switches must match this assumption. The servo is represented as an A axis so its commands are queued with the translation moves.

Axis or output	Step GPIO	Direction GPIO	Configuration
X motor 1	13	14 inverted	80 steps/mm
X motor 2	16	18 inverted	80 steps/mm
Y	27	26	80 steps/mm
Z	25	33	254.650 steps/mm
Gripper A	19 PWM	Not applicable	50 Hz, 1000–2000 μ s

Distance calibration

For the belt-driven axes, a 20-tooth pulley and 2 mm belt pitch produce 40 mm of travel per revolution. Dividing 3200 pulses by 40 mm gives the configured 80 steps/mm. The Z-axis configuration records a calibration correction: a 100-unit command initially produced 20 mm of movement, so the previous 50.930 steps/mm value was multiplied by five to obtain 254.650 steps/mm. This accounts for the reduction in the assembled vertical drive.

The configured maximum rates are 2000 mm/min for X and 4000 mm/min for Y and Z; accelerations are 300 mm/s² for the horizontal axes and 200 mm/s² for Z. These are command limits, not measured average transfer speeds. Acceleration, simultaneous-axis motion, servo dwell and the length of each transfer affect the actual duration.

Startup and protective limits

The built robot has no limit switches and uses manual origin positioning. The reviewed configuration accordingly disables homing, hard limits and soft limits. The vertical carriage is placed at its upper reference before reset or startup. The team stops the robot in an emergency by cutting power; a dedicated emergency-stop circuit was not reported.

Two integration details are documented in the repository. The X step output was moved from GPIO12 to GPIO13 because the original wiring affected an ESP32 boot-strapping pin. The servo configuration also requires the recognized pwm_hz key; an invalid firmware configuration can otherwise prevent the expected machine setup from loading. Reading the controller startup log is therefore part of configuration verification.

7 Speech input and chess logic

VoiceChess uses Vosk for offline speech recognition, with a USB microphone supplying mono audio at 16 kHz. The recognized transcript is interpreted as a chess move rather than passed directly to the motion controller. The system accepts square-based phrases and supported piece or castling forms and normalizes common spoken aliases for board coordinates.

Matching and confirmation

The voice module compares candidate phrases using normalized Levenshtein similarity through RapidFuzz. It ranks matches by move rather than simply choosing the best individual phrase. The configured acceptance threshold is 0.90, and the best move must exceed its nearest rival by at least 0.05. These values are matching thresholds, not measured speech-recognition accuracy.

The current legal moves provide a position-dependent candidate set. A broader pool of move-shaped phrases also lets the module distinguish a confidently recognized illegal move from unrecognized speech. Unknown tokens, weak matches and near ties are rejected. A bare castling request is clarified when the side is ambiguous. This is important because similar-sounding file letters can otherwise turn a plausible transcript into the wrong move.

Spoken confirmation is enabled in the live voice path: the system reads back the interpreted move and waits for a yes or no response. Rejected or unclear confirmations lead to another attempt rather than immediate motion. Text mode supports development without live recognition and does not provide evidence about microphone performance. The combined legal filtering and confirmation reduce opportunities for unintended moves, but do not establish a quantified recognition error rate.

Rules and computer opponent

The chess_ai package uses python-chess as the rules authority. It maintains the board, returns legal UCI moves, validates attempted moves and reports game status. Stockfish supplies the computer's reply through its engine interface. The default configuration uses skill level 5 and a 0.5-second thinking allowance; the command-line interface permits different values. The default orchestration loop also has a 40-turn limit, which is an application setting rather than a chess rule.

Human-readable move feedback is generated from the rules engine's result. The speech layer supports available local speech backends, including espeak-ng, with fallbacks. The normal arrangement assigns White to the human and Black to the computer. Offline processing avoids a cloud speech dependency, although the models, engine and other dependencies must first be installed on the Raspberry Pi.

8 Motion planning and special moves

The planner converts algebraic squares into machine coordinates using 58 mm square spacing and the center of a1 as the origin. X increases with rank, and Y increases with file. Thus, e2 maps to X58 Y232, while e4 maps to X174 Y232. This explicit convention avoids the older axis-label ambiguity in the presentation. The configured usable horizontal envelope is X0–535 mm and Y0–545 mm.

Piece	Pickup depth Z in mm	Servo closing angle A
Pawn	85	62°
Knight	75	76°
Bishop	80	43°
Rook	90	43°
Queen	67	35°
King	67	35°

A transfer raises the empty gripper, moves above the source square, lowers to the calibrated pickup depth, closes and waits, lifts the piece, follows the transfer path, lowers to the destination, opens and retracts. The configured loaded feed is 2000 mm/min and the gripper dwell is 0.5 seconds. Ordinary unobstructed moves can use a 15 mm lift. High transfers use the upper Z position and route around protected king and queen squares.

This routing is necessary because the 115 mm working clearance does not guarantee that every held piece clears the tallest neighbours. The shallowest pickup profile leaves only about 67 mm beneath the carried piece at the upper position, just 2 mm above a configured 65 mm bishop. The planner therefore treats kings and queens as protected obstacles and constructs a path around them. These geometric checks still depend on accurate alignment and the assumed piece dimensions.

Captures and compound moves

For a capture, the victim is moved to an off-board storage slot before the attacking piece is transferred. Sixteen configured slots form an L-shaped storage area, with 62 mm spacing. Castling moves the king first, then transfers the rook with the king's new position included in obstacle avoidance. En passant removes the pawn from its actual occupied square rather than the destination square.

Promotion removes the pawn and fetches a reserve queen for the appropriate colour. There is one configured queen reserve per colour. Although chess logic can represent underpromotion, the reviewed physical routine still selects a queen for rook, bishop or knight promotion requests and emits a warning. Underpromotion is therefore not physically supported. En passant and promotion remain untested on the real robot; finite storage and reserve capacity also require operator intervention when exhausted.

9 Integration and evaluation

The completed prototype brings speech entry, rules validation, computer play and physical transfers into a single turn loop. The team reports that the integrated functions work, including ordinary moves, captures and castling. The presentation contains recordings of calibration, piece movement, the beginning of a game, bishop and knight movement, and kingside castling. En passant and promotion have not been tested on the robot.

Function	Evidence and validation status
Voice controlled play	Reported working by the team; no recognition benchmark supplied
Ordinary moves and captures	Reported working on the robot
Castling	Reported working; kingside castling recording included in the presentation
En passant	Implemented; not tested on the robot
Queen promotion	Implemented; not tested on the robot
Underpromotion	Rules representation exists; physical routine uses a queen instead
Complete game reliability	No counted game series or failure-rate measurements supplied

10 Limitations and further development

The main remaining engineering work concerns fault detection, protective limits and broader validation. These needs follow directly from the current open-loop architecture and the finite physical resources around the board.

Machine protection and repeatable startup

Limit switches, a repeatable homing procedure and a dedicated, verified emergency-stop arrangement would reduce dependence on manual setup and power disconnection. The present firmware keeps motors energized between moves but does not provide automatic position recovery.

Clearance and piece handling

Greater clearance and gripper compliance would improve tolerance to piece-placement errors. Capture storage and spare-piece capacity need explicit monitoring, with a controlled pause before either is exhausted. Physical underpromotion requires the requested replacement pieces and a planner that selects them.

Voice and accessibility evaluation

Recognition needs tests with different speakers, microphone positions and background noise. Evaluation with intended users should also measure the effort of correcting misunderstood moves. The prototype demonstrates the input concept, but does not establish accessibility across users or environments.

VoiceChess meets the central prototype objective of converting spoken chess moves into physical piece transfers with an offline computer opponent. The mechanical design was practical to manufacture with the available tools, and the software separates rules, interpretation and motion execution clearly. The next milestone is systematic physical validation, particularly for special moves and recovery, followed by repeatable homing and verified protective controls.